



**National Institute of Electronics and Information Technology,
Chennai**

Online Internship in RTL Design using Verilog HDL

Project Report

on

**Project Report: Verilog HDL Implementation of AXI4-
Lite Interface Controller**

Submitted By

Chella jeevana jyothi

at

July 2026 – August 2026

About NIELIT

National Institute of Electronics and Information Technology (NIELIT), headquartered in New Delhi is the capacity building arm of the Ministry of Electronics and Information Technology (MeitY), mandated to carry out HR development and related activities in Information, Electronics and Communication Technology (IECT).

Since its inception in 2010, NIELIT Chennai has established itself as a premier institution providing affordable quality education as per the job market requirements for candidates primarily from Tamil Nadu. As of date NIELIT Chennai has imparted skill training to more than 30, 000 youth of this region and conducted digital literacy certification for more than 25,000 candidates. NIELIT Chennai handles the activities of the NIELIT in Tamil Nadu, Andhra, Telangana, and the Andaman and Nicobar Islands.

Presently NIELIT Chennai is implementing the following projects for Tamil Nadu: ISEA (Information Security Education and Awareness), Future Skills PRIME capacity building projects (3D Printing and Additive Manufacturing, Robotic Process Automation, IoT - technology streams), and the aspirational district projects for SC/ST candidates at Ramanathapuram and Virudhunagar funded by MeitY, Government of India. The centre also received funding through the electronics product design and product testing capacity building project funded by MeitY.

Other than training, the centre also provides services like; Private Cloud Setup using Citrix, vSphere, Red Hat & FOSS Cloud, Hyper-converged Infrastructure (HCI) and Virtual Desktop Infrastructure (VDI) Server Setup, High-Performance Computing (HPC) Setup with 100G/200G connectivity, Server and Virtual Lab Setup for BigData, AI & Information Security, and Digital & Mobile Forensics Service.

NIELIT has also set up a Virtual Academy, a common End-to-End Platform to conduct Online training courses including a virtual lab environment in the areas of IECT from foundation to expert level targeting from school students to working professionals.

INDEX

Contents	Page No.
ACKNOWLEDGEMENT	
ABSTRACT	
LIST OF FIGURES	
1.PROJECT OVERVIEW	1
2. Verilog Code Design	2
3. Behavioral Simulation	4
4.Synthesis 4.1Post- Synthesis Functional Simulation 4.2 Post- Synthesis Timing Simulation	5
5. Static Timing Analysis (STA)	6
6. Implementation Process 6.1 Post- Implementation Functional Simulation 6.2 Post- Implementatio	7
7. Summary	

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to all those who have supported and guided me throughout the completion of this project on __21st August 20026__ as part of the Online Internship in RTL Design using Verilog HDL, conducted by the National Institute of Technology and Applied Research (NIELIT), Chennai.

First and foremost, I would like to thank my course instructor, Ishant Kumar Bajpai (Scientist 'D') , for their invaluable guidance, constructive feedback, and constant encouragement throughout the project. Their expertise in the field of Chip Design has been a key factor in the successful completion of this report.

I would also like to extend my appreciation to the NIELIT Chennai for offering this comprehensive online course, providing us with high-quality learning resources and support despite the challenges of remote learning.

A special thanks to my fellow participants in the online course for the engaging discussions and knowledge-sharing that enriched my learning experience. The collaborative nature of this course has significantly enhanced the depth and quality of my work.

Finally, I would like to thank all the authors, researchers, and resources referenced in this report for their contributions to the field of Chip Design, which formed the foundation of my project.

Thank you to everyone who has been part of this learning journey.

Project Report: Verilog HDL Implementation of AXI4-Lite Interface Controller

1. Project Overview

This project implements an **ARM AMBA AXI4-Lite** slave interface controller in **Verilog HDL**. The AXI4-Lite protocol is a lightweight subset of the AXI4 standard, intended for control-register access without burst transfers. The controller acts as an IP core within an FPGA design, allowing an external master to read/write internal configuration registers via the standard AXI bus.

2. Verilog Code Design

Module Functionality: A complete AXI4-Lite slave that correctly handles both **read** and **write** transactions from a master.

Interface Signals: The module implements the **five independent channels** defined by the AXI4-Lite protocol. All channels use the **VALID/READY handshake** mechanism.

Channel	Signals
Write Address (AW)	AWADDR, AWVALID, AWREADY
Write Data (W)	WDATA, WSTRB, WVALID, WREADY

Channel	Signals
Write Response (B)	BRESP, BVALID, BREADY
Read Address (AR)	ARADDR, ARVALID, ARREADY
Read Data (R)	RDATA, RRESP, RVALID, RREADY
Global	ACLK (clock), ARESETN (active- low async reset)

Core Architecture: A **finite-state machine (FSM)** manages handshaking and transaction sequencing across channels. The module operates on the **rising edge** of **ACLK** while **ARESETN** is **high**. The code skeleton is as follows:

```
module axi_lite_slave #(
    parameter ADDR_WIDTH = 4,
    parameter DATA_WIDTH = 32
)(
    input wire ACLK,
    input wire ARESETN,
    // Write address channel
    input wire [ADDR_WIDTH-1:0] AWADDR,
    input wire AWVALID,
    output reg AWREADY,
    // Write data channel
    input wire [DATA_WIDTH-1:0] WDATA,
```

```

input wire [3:0] WSTRB,
input wire WVALID,
output reg WREADY,
// Write response channel
output reg [1:0] BRESP,
output reg BVALID,
input wire BREADY,
// Read address channel
input wire [ADDR_WIDTH-1:0] ARADDR,
input wire ARVALID,
output reg ARREADY,
// Read data channel
output reg [DATA_WIDTH-1:0] RDATA,
output reg [1:0] RRESP,
output reg RVALID,
input wire RREADY
);
// Internal registers and FSM logic
// ...
Endmodule

```


3. Behavioral Simulation

Purpose: Verify the functional correctness of the RTL code **without any timing information**.

Procedure:

1. Create a Vivado RTL project, adding the design source files and a testbench.
2. In the **Flow Navigator**, click **Run Simulation > Run Behavioral Simulation**.
3. The Vivado Simulator launches; observe waveforms to validate the handshake sequences and data paths for both read and write transactions, ensuring protocol compliance.

4. Synthesis

Purpose: Translate the RTL code into a **gate-level netlist** optimised for the target FPGA (e.g., Xilinx 7- series).

After synthesis, the following two simulations are performed:

4.1 Post- Synthesis Functional Simulation

Objective: Verify that **logic optimisations** (e.g., constant propagation, resource sharing) introduced during synthesis have **not altered** the original functionality.

Action: In the Flow Navigator, click **Run Simulation > Post- Synthesis Functional Simulation**.

Note: This simulation uses the **UNISIM** primitive library and **does not** include timing delays.

4.2 Post- Synthesis Timing Simulation

Objective: Obtain **early timing estimates** using the synthesised netlist with **estimated wire- load models**, identifying potential critical paths.

Action: In the Flow Navigator, click **Run Simulation > Post- Synthesis Timing Simulation**.

Note: This simulation generates a Standard Delay Format (SDF) file with **estimated delays** and back- annotates it to the netlist.

5. Static Timing Analysis (STA)

Purpose: **Exhaustively** analyse all paths in the design (without test vectors) to verify **setup** and **hold** time constraints under worst- case conditions (process, voltage, temperature corners).

Procedure: After synthesis or implementation, Vivado automatically constructs a **Timing Graph** as the basis for STA.

In the **Flow Navigator**, click **Report Timing Summary** to generate the timing report.
Key metrics to analyse:

WNS (Worst Negative Slack) – the most negative setup slack.

WHS (Worst Hold Slack) – the most negative hold slack.

Check path types (reg2reg, reg2pin, pin2reg, etc.) for both setup and hold violations against the applied constraints.

6. Implementation Process

Purpose: Perform **Place & Route** (P&R) on the synthesised netlist to produce the final physical design file.

After implementation, the following two simulations are executed:

6.1 Post- Implementation Functional Simulation

Objective: Verify that **physical optimisations** (e.g., clock- tree synthesis, retiming) performed during P&R do **not break** functional behaviour.

Action: In the Flow Navigator, click **Run Simulation > Post- Implementation Functional Simulation**.

Note: This simulation still uses the **UNISIM** library and **excludes** timing information.

6.2 Post- Implementation Timing Simulation

Objective: Use the **most accurate** actual delays from the routed design – this simulation is **closest to real silicon behaviour**.

Action: In the **Project Manager**, click **Run Simulation > Run Post- Implementation Timing Simulation**.

Note: An SDF file with **actual P&R delays** is generated and back- annotated. The simulation results confirm whether the final design operates reliably at the target clock frequency.

Note: An SDF file with **actual P&R delays** is generated and back-annotated. The simulation results confirm whether the final design operates reliably at the target clock frequency.

7. Summary

This project successfully designed and verified an AXI4- Lite slave interface controller. By progressing from **behavioural simulation** through **post- synthesis** and **post- implementation** simulations, and by performing a thorough **static timing analysis**, both **functional correctness** and **timing robustness** have been ensured. The design meets all specified requirements and is ready for integration into a larger FPGA system.